

PDU Protocol ICD

Purpose

This document defines the fresh framed UART protocol for the Artemis PDU MCU.

This protocol replaces the older newline-terminated ASCII-offset packet scheme. Backward compatibility with the old protocol is intentionally not preserved.

Hardware context comes from the Artemis PDU manual and in-repo hardware notes. The temporary student PDF ICD is not the current source of truth for firmware or controller-side implementation.

Design Goals

- Robust UART framing
- Explicit request/response transactions
- CRC-protected payloads
- Small fixed-width binary fields only
- Mission-facing logical outputs instead of raw GPIO choreography
- Enough functionality for demo bring-up, output control, and SOH visibility

Quick Glossary

- `SOF`: start of frame. A marker byte that tells the receiver where a new message begins.
- `CRC`: cyclic redundancy check. A checksum used to detect corrupted bytes.
- `opcode`: operation code. The specific command being requested.
- `seq`: sequence number. Chosen by the host and echoed by the PDU in the response.
- `payload`: the command-specific data bytes inside the frame.
- `ICD`: interface control document. The formal description of the wire protocol.

Transport

- Physical link: UART
- Baud rate: fixed 9600 baud
- Byte order: little-endian for multi-byte numeric payload fields
- Framing: fixed start-of-frame byte plus explicit payload length
- Integrity check: CRC-16/CCITT over header and payload

Frame Format

Each frame is:

Byte	Field	Size	Notes
0	sof	1	Always 0xA5
1	version	1	Currently 0x02
2	msg_type	1	0=request, 1=response, 2=event
3	opcode	1	Command or event ID
4	seq	1	Sequence number echoed in the response
5	status	1	In requests set to 0; in responses contains the status code
6	payload_len	1	Number of payload bytes
7..N	payload	variable	payload_len bytes
N+1..N+2	crc16	2	CRC-16/CCITT over bytes 1..N

Notes:

- sof is not included in the CRC.
- Once a frame starts, the parser uses payload_len to determine frame length. 0xA5 may appear inside payload or CRC bytes.
- An incomplete frame is discarded if no next byte arrives within 100 ms.
- The current firmware only emits response frames. event is reserved for later use.

Status Codes

Value	Name	Meaning
0x00	OK	Request accepted and processed
0x01	BAD_OPCODE	Unknown opcode
0x02	BAD_LENGTH	Payload length did not match the opcode contract
0x03	BAD_PARAM	Payload field value was invalid
0x04	HW_FAULT	Reserved for hardware-side command rejection
0x05	NOT_IMPLEMENTED	Reserved for future use
0x06	BUSY	A timed operation is already active

Output IDs

The protocol exposes logical outputs, not raw pin names.

ID	Name	Meaning
0x01	PDU_OUTPUT_3V3_1	3.3 V rail 1
0x02	PDU_OUTPUT_3V3_2	3.3 V rail 2
0x03	PDU_OUTPUT_5V_1	5 V rail 1
0x04	PDU_OUTPUT_5V_2	5 V rail 2
0x05	PDU_OUTPUT_5V_3	5 V rail 3
0x06	reserved	5 V input to the 12 V switch regulator
0x07	PDU_OUTPUT_12V	12 V logical rail
0x08	PDU_OUTPUT_VBATT	Battery bus enable
0x09	PDU_OUTPUT_BURN1	Burn-wire deployment channel 1
0x0A	PDU_OUTPUT_BURN2	Burn-wire deployment channel 2
0x0B	reserved	Coarse H-bridge group ID; use torque-coil commands
0x0C	reserved	Coarse H-bridge group ID; use torque-coil commands
0xFF	PDU_OUTPUT_ALL	All-output readback target

Important implementation notes:

- PDU_OUTPUT_12V maps to both SW_5V_EN4 and SW_12V_EN1.
- SW_5V_EN4 is reserved as the 5 V input to the 12 V switch regulator and is not independently commandable in this MVP protocol.
- PDU_OUTPUT_BURN1 and PDU_OUTPUT_BURN2 are separate deployment burn channels that both depend on shared BURN_5V.
- H-bridge torque control is exposed only through SET_TORQUE_COIL and GET_TORQUE_COIL.

Supported Opcodes

0x00 PING

Request payload:

- none

Response payload:

Byte	Field
0	protocol version

This is the simplest link-alive check. It does not touch hardware state.

0x01 GET_PROTOCOL_INFO

Request payload:

- none

Response payload:

Byte	Field
0	protocol version
1	capability bitmap
2	max payload length
3	output count
4	firmware major
5	firmware minor
6	firmware patch

Capability bitmap:

- bit 0: CRC-16 present
- bit 1: sequence/ack pattern supported
- bit 2: reset info supported
- bit 3: summary status supported
- bit 4: power-cycle output supported
- bit 5: software reset through watchdog stall supported
- bit 6: help response supported
- bit 7: actuator command support is present; see `FIRE_BURN_WIRE` and torque-coil commands

`0x02 GET_SUMMARY_STATUS`

Request payload:

- none

Response payload:

Byte	Field
0..1	output enable bitmap
2	reset cause (<code>RSTC_RCAUSE</code>)
3	fault bitmap
4..7	uptime seconds
8	capability bitmap

Current implementation notes:

- `fault bitmap` reports active H-bridge fault indications.
- `uptime seconds` is derived from the FreeRTOS scheduler tick and accumulated across 16-bit tick rollover.
- The output bitmap bit order matches the public output order below, excluding reserved IDs `0x06`, `0x0B`, `0x0C`, and `0xFF`.

Fault bitmap:

- bit 0 = H-bridge 1 fault indication active
- bit 1 = H-bridge 2 fault indication active

Bitmap order:

- bit 0 = 3V3_1
- bit 1 = 3V3_2
- bit 2 = 5V_1
- bit 3 = 5V_2
- bit 4 = 5V_3
- bit 5 = 12V
- bit 6 = VBATT
- bit 7 = BURN1
- bit 8 = BURN2

0x03 GET_RESET_INFO

Request payload:

- none

Response payload:

Byte	Field
0	reset cause (RSTC_RCAUSE)

Reset cause bit meanings:

- bit 0: power-on reset
- bit 1: core brownout reset
- bit 2: VDD brownout reset
- bit 4: external reset
- bit 5: watchdog reset
- bit 6: CPU system reset request

SOFTWARE_RESET intentionally stops watchdog service. After reboot, GET_RESET_INFO should report the hardware source that actually reset the MCU, usually external reset if the external watchdog fires first, or watchdog reset if the internal SAME51 watchdog fires first.

0x04 HELP

Request payload:

- none

Response payload:

- ASCII command/version summary

0x10 GET_OUTPUT_STATE

Request payload:

Byte	Field
0	output ID or 0xFF for all

Response payload for a single output:

Byte	Field
0	output ID
1	applied/measured state (0 or 1)

Response payload for PDU_OUTPUT_ALL :

Byte	Field
0	0xFF
1	output count
2..10	output states in bitmap/output-ID order

0x11 SET_OUTPUT_STATE

Request payload:

Byte	Field
0	output ID or 0xFF for all
1	desired state (0=disable, 1=enable)

Response payload:

- same shape as GET_OUTPUT_STATE

This opcode responds with post-command state, not just an ACK bit. 0xFF / PDU_OUTPUT_ALL is rejected for SET_OUTPUT_STATE ; all-output access is read-only in this protocol so there is no one-command enable-all path. Burn-wire outputs are also rejected by SET_OUTPUT_STATE ; use FIRE_BURN_WIRE so deployment channels are always time-bounded. Reserved H-bridge IDs are rejected by SET_OUTPUT_STATE ; use SET_TORQUE_COIL so the host can command a specific coil direction/current state.

0x12 POWER_CYCLE_OUTPUT

Request payload:

Byte	Field
0	output ID
1..2	off time in milliseconds, little-endian

Response payload:

Byte	Field
0	output ID
1	applied/measured state after starting the power cycle

Rules:

- off time must be `1..10000` ms
- output must already be on
- `PDU_OUTPUT_ALL` is rejected
- reserved `0x06`, burn channels, and H-bridge outputs are rejected
- if another timed operation is active, the command returns `BUSY`

The command is nonblocking. Firmware turns the output off, replies immediately, then restores the output after the requested off time from the app task service loop. During the active timed operation, generic output set commands return `BUSY`.

`0x13 FIRE_BURN_WIRE`

Request payload:

Byte	Field
0	output ID, <code>PDU_OUTPUT_BURN1</code> or <code>PDU_OUTPUT_BURN2</code>
1..2	fire time in milliseconds, little-endian
3..4	arm token, little-endian, must be <code>0xB142</code>

Response payload:

Byte	Field
0	output ID
1	applied/measured state after starting the burn pulse

Rules:

- fire time must be `1..10000` ms
- only `PDU_OUTPUT_BURN1` and `PDU_OUTPUT_BURN2` are accepted
- the arm token must match exactly
- if any burn channel or timed operation is already active, the command returns `BUSY`

The command is nonblocking and time-bounded. Firmware enables the shared burn source, enables the selected burn channel, replies, and then forces the selected burn channel off after the requested duration.

`0x14 SET_TORQUE_COIL`

This is a low-level PDU interface for the OBC/Teensy ADCS controller. The PDU does not run attitude-control algorithms; it only maps validated coil commands to the DRV8847 H-bridge pins.

Request payload:

Byte	Field
0	coil ID, 1..4
1	mode, 0=coast/off, 1=forward, 2=reverse, 3=brake
2	current scalar, 0=100%, 1=50%
3..4	duration in milliseconds, little-endian; 0=latch until changed

Response payload:

Byte	Field
0	coil ID
1	measured/applied mode
2	current scalar for that coil's driver
3	driver awake state, 1=awake, 0=sleep
4	fault active, 1=fault, 0=no fault

Coil mapping:

- coil 1: DRV8847 U1 bridge 1/2, IN1/IN2
- coil 2: DRV8847 U1 bridge 3/4, IN3/IN4
- coil 3: DRV8847 U2 bridge 1/2, IN5/IN6
- coil 4: DRV8847 U2 bridge 3/4, IN7/IN8

Mode mapping, using each coil's INA/INB pair:

- coast/off: 0/0
- forward: 1/0
- reverse: 0/1
- brake: 1/1

Rules:

- duration must be 0..60000 ms
- if duration is nonzero, the command is nonblocking and the coil is returned to coast/off when the duration expires
- burn and power-cycle timed operations block torque commands and return BUSY
- a timed torque command can be replaced or cancelled by another command for the same coil, including an immediate coast/off command
- only one timed torque command can be tracked at a time; a duration command for a different coil while one is active returns BUSY
- TRQ1 is shared by coils 1 and 2; TRQ2 is shared by coils 3 and 4
- if the sibling coil on the same driver is active, a command that would change the shared current scalar returns BAD_PARAM
- nSLEEP is held high while either coil on that driver is active and returned low only after both coils are coast/off

0x15 GET_TORQUE_COIL

Request payload:

Byte	Field
0	coil ID, 1..4

Response payload:

- same shape as SET_TORQUE_COIL

0x16 GET_CHARGER_STATUS

Request payload:

- none

Response payload:

Byte	Field
0	charger enabled state, 1=enabled, 0=shutdown
1	charge indicator active, 1=CHRG asserted low, 0=not asserted
2	SHDN output latch level
3	raw CHRG pin level

LTC4012 polarity:

- SHDN high enables/allows charger operation
- SHDN low shuts the charger down
- CHRG is active-low/open-drain

The digital CHRG readback reports whether the indicator is low or high. Board pull-up behavior and the LTC4012 weak-pulldown/high-Z states should be considered before treating this as detailed charger state telemetry.

0x17 SET_CHARGER_STATE

Request payload:

Byte	Field
0	desired charger state, 1=enable, 0=shutdown

Response payload:

- same shape as GET_CHARGER_STATUS

This command controls only the LTC4012 SHDN input. It does not validate solar input, battery temperature, or battery charge current.

0x20 SOFTWARE_RESET

Request payload:

- none

Response payload:

- none

The firmware replies `OK`, then asks the watchdog task to stop servicing both the external watchdog pin and the internal SAME51 watchdog. This causes a real hardware reset path instead of only jumping through a software reset routine.

Current Firmware Scope

The current firmware implements:

- protocol framing and CRC validation
- request/response sequencing
- protocol info query
- ping query
- help query
- summary status query
- reset cause query
- single-output state query
- all-output state query
- single-output state set for normal latchable outputs
- nonblocking single-output power cycle
- timed burn-wire fire command
- low-level torque-coil set/read interface for external ADCS control
- charger enable/status control through `SHDN` and `CHRG`
- software reset through watchdog stall

The current firmware does not yet implement:

- asynchronous event frames
- Pi-specific power/reset supervision commands beyond the PDU watchdog reset
- latched-fault reporting beyond live H-bridge fault bits
- heater abstractions separate from burn-wire channels
- all-output enable command

Bench profiles such as `ALL`, `VIBE`, and `THERMAL` should be implemented as separate Teensy-side test sketches, not as PDU firmware opcodes.

Error Handling Rules

- A valid request with an unknown opcode returns `BAD_OPCODE`.
- A valid request with the wrong payload length returns `BAD_LENGTH`.
- A valid request with an invalid output ID or invalid state value returns `BAD_PARAM`.
- `SET_OUTPUT_STATE` with `PDU_OUTPUT_ALL` returns `BAD_PARAM`.
- `SET_OUTPUT_STATE` with a burn-wire output returns `BAD_PARAM`.

- `GET_OUTPUT_STATE` or `SET_OUTPUT_STATE` with reserved H-bridge IDs returns `BAD_PARAM`.
- Commands that conflict with an active timed operation return `BUSY`.
- A frame with bad CRC is ignored.
- A frame with the wrong protocol version returns `BAD_PARAM`.
- A frame with a message type other than `request` is ignored by the current firmware.

Example Transactions

Example: `GET_PROTOCOL_INFO`

Request:

- `msg_type = 0`
- `opcode = 0x00`
- `seq = 0x10`
- `status = 0x00`
- `payload_len = 0`

Response:

- `msg_type = 1`
- `opcode = 0x00`
- `seq = 0x10`
- `status = 0x00`
- payload contains protocol version

Example: `GET_PROTOCOL_INFO`

Request:

- `msg_type = 0`
- `opcode = 0x01`
- `seq = 0x11`
- `status = 0x00`
- `payload_len = 0`

Response:

- `msg_type = 1`
- `opcode = 0x01`
- `seq = 0x11`
- `status = 0x00`
- payload contains protocol and firmware metadata

Example: Enable `5V_1`

Request payload:

- byte 0 = `0x03`

- byte 1 = `0x01`

Response payload:

- byte 0 = `0x03`
- byte 1 = measured state after command

Example: Read all output states

Request payload:

- byte 0 = `0xFF`

Response payload:

- byte 0 = `0xFF`
- byte 1 = `9`
- bytes 2..10 = the 9 current output states

Implementation References

- Protocol constants: [src/pdu_protocol_v2.h](#)
- Parser and handlers: [src/pdu_packet.c](#)
- UART byte ingress path: [src/app.c](#)

Migration Guidance For The Controller Side

- Treat this ICD as the source of truth for the new protocol.
- Do not reuse the old ASCII-offset packet logic.
- Implement CRC validation and sequence matching on the host side.
- Start by integrating these calls in order:
 1. `GET_PROTOCOL_INFO`
 2. `GET_SUMMARY_STATUS`
 3. `SET_OUTPUT_STATE`
 4. `GET_OUTPUT_STATE`
 5. `POWER_CYCLE_OUTPUT`
 6. `GET_CHARGER_STATUS`
 7. `SET_CHARGER_STATE`
 8. `FIRE_BURN_WIRE`, only after deployment safety policy is agreed